

Fondamentaux des API avec Python

FastAPI · Pydantic · REST · OpenAPI · Swagger UI

Objectifs

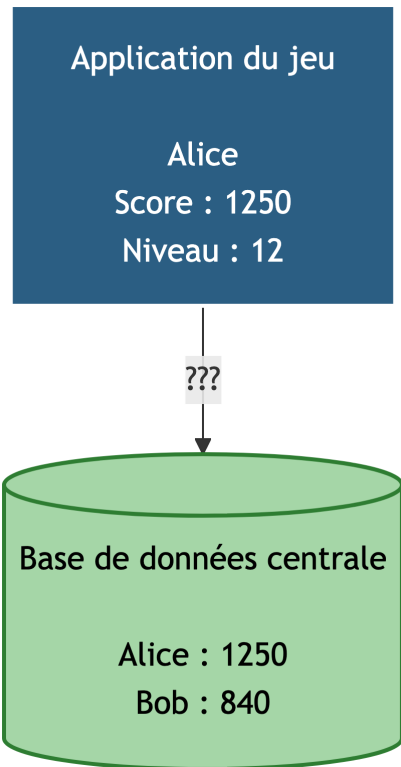
À la fin de cette séance, vous serez capables de :

- comprendre ce qu'est une API ;
 - expliquer le fonctionnement d'une communication client / serveur ;
 - comprendre les principes fondamentaux d'une API REST ;
 - utiliser les principales méthodes HTTP ;
 - créer une API REST avec **FastAPI** ;
 - définir des routes et leurs paramètres ;
 - recevoir et retourner des données JSON ;
 - créer des modèles avec **Pydantic** ;
 - valider automatiquement les données reçues ;
 - définir le contrat de réponse d'une API ;
 - comprendre le rôle d'**OpenAPI** ;
 - utiliser **Swagger UI** pour tester une API ;
 - comprendre comment FastAPI génère automatiquement sa documentation.
-

1. Pourquoi avons-nous besoin d'APIs ?

Imaginez un jeu vidéo sur smartphone. L'application doit afficher le score et le niveau de la joueuse, Alice.

Ces informations sont stockées sur un serveur distant, dans une **base de données**.



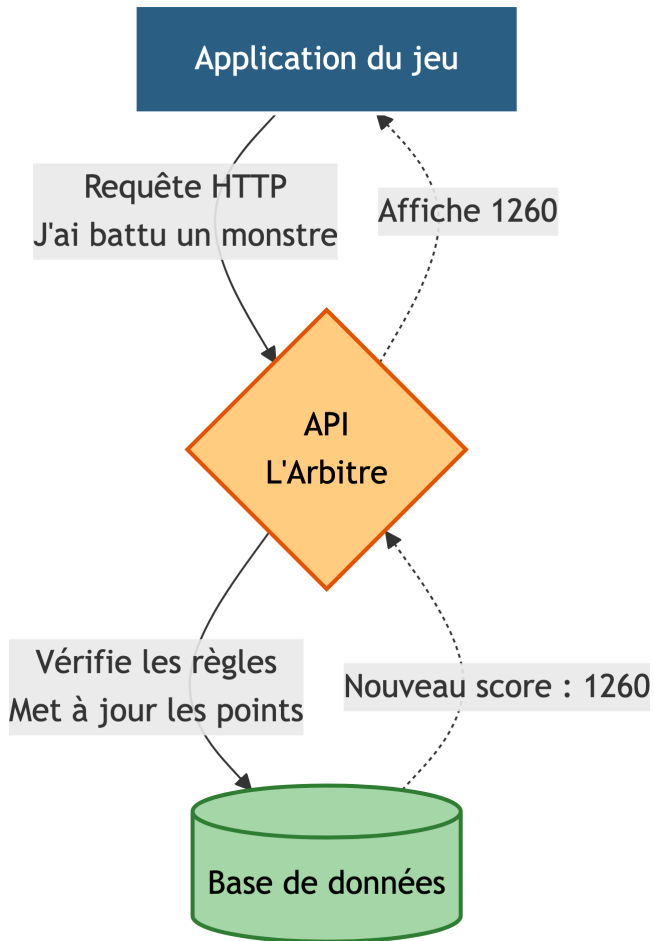
L'application de votre téléphone pourrait-elle **se connecter directement** à cette **base de données** pour lire et modifier les scores ?

Techniquement, oui. Mais ce serait une très mauvaise idée.

Si l'application communique directement avec la base de données, cela signifie que le téléphone contient les codes d'accès à cette base. Un joueur un peu bricoleur pourrait intercepter la connexion et dire à la base de données : *“Modifie mon score et mets 999 999 points”*.

De plus, l'application devrait calculer elle-même si le joueur a le droit de passer au niveau supérieur, ce qui rend les mises à jour complexes si l'on sort le jeu sur d'autres supports.

C'est pour cela que l'on intercale un **arbitre** : l'API.



L'application ne parle plus à la base de données. Elle envoie une requête à l'API : *"Alice vient de battre un monstre"*.

L'API, qui contient toutes les règles du jeu, vérifie que l'action est valide, calcule les points gagnés, met elle-même la base de données à jour en toute sécurité, puis renvoie le nouveau score au téléphone.

Cette couche intermédiaire est une **API**.

2. Qu'est-ce qu'une API ?

API signifie :

Application Programming Interface

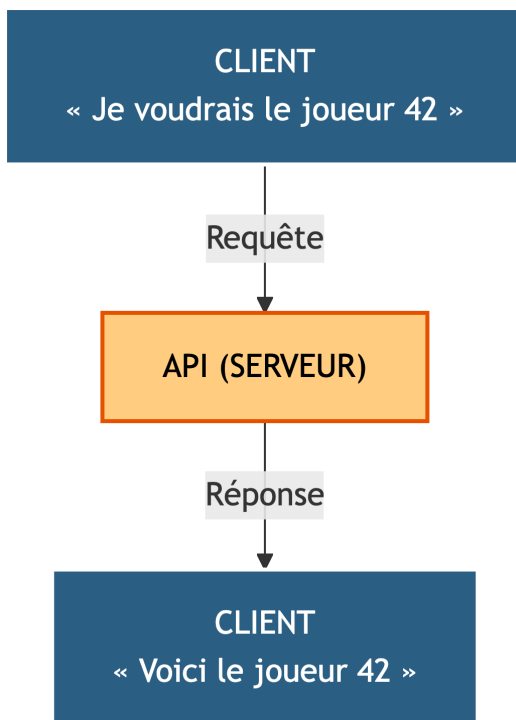
Une API est une **interface permettant à un programme d'interagir avec un autre programme ou service**.

L'API définit notamment :

- comment demander une information ;
 - quelles informations envoyer ;
 - comment les données sont structurées ;
 - quelles réponses sont possibles ;
 - quelles erreurs peuvent être retournées.
-

Une API comme contrat

On peut voir une API comme un contrat entre deux applications.



Le client n'a pas besoin de connaître :

- le langage utilisé par le serveur ;

- la base de données ;
- l'architecture interne ;
- le code du serveur.

Il doit simplement connaître **le contrat de l'API**.

3. Exemple : utiliser une API existante

Python peut communiquer avec une API grâce à HTTP.

Par exemple :

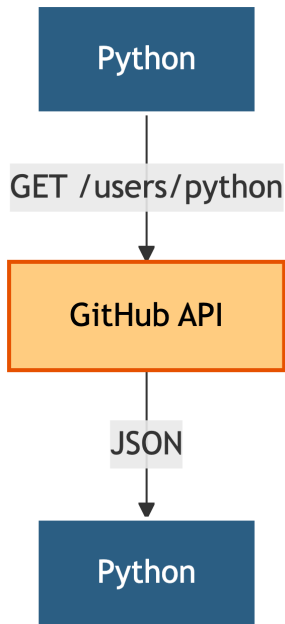
```
import requests

response = requests.get(
    "https://api.github.com/users/python"
)

print(response.status_code)
print(response.json())
```

La fonction `requests.get()` envoie une requête HTTP.

Le serveur retourne une réponse.



Que contient une réponse HTTP ?

Une réponse possède notamment :

- un **code de statut** ;
- des **en-têtes** ;
- un **contenu**.

Exemple :

```
HTTP/1.1 200 OK

Content-Type: application/json

{
  "login": "python",
  "id": 1525981
}
```

4. Les codes HTTP

Quelques codes sont particulièrement importants.

Code	Signification
200	La requête a réussi
201	Une ressource a été créée
204	Succès, sans contenu à retourner
400	Requête incorrecte
401	Authentification nécessaire
403	Accès interdit
404	Ressource inexistante
422	Données reçues invalides
500	Erreur interne du serveur

Une API ne doit donc pas seulement retourner des données.

Elle doit également indiquer **ce qui s'est passé**.

5. Le format JSON

Les APIs REST utilisent très souvent le format **JSON**.

JSON permet de représenter des données structurées.

```
{  
  "id": 42,  
  "name": "Alice",  
  "score": 1250  
}
```

Un objet JSON peut contenir :

- des chaînes de caractères ;
- des nombres ;
- des booléens ;
- des tableaux ;
- d'autres objets.

Par exemple :

```
{
  "id": 42,
  "name": "Alice",
  "score": 1250,
  "weapons": [
    "sword",
    "bow"
  ],
  "active": true
}
```

6. API REST

REST signifie :

Representational State Transfer

REST est un ensemble de principes permettant notamment d'organiser une API autour de **ressources**.

Dans notre jeu, les ressources pourraient être :

```
players
games
weapons
scores
```

On peut alors imaginer :

```
GET    /players
GET    /players/42

POST   /players

PUT    /players/42

DELETE /players/42
```


7. Les méthodes HTTP

Les quatre méthodes principales que nous utiliserons sont :

Méthode	Action
GET	Lire
POST	Créer
PUT	Modifier
DELETE	Supprimer

On peut les rapprocher des opérations CRUD :

HTTP	CRUD
GET	Read
POST	Create
PUT	Update
DELETE	Delete

8. Notre fil rouge : Game API

Nous allons construire progressivement une API pour un petit jeu.

Notre API devra gérer des joueurs.

Un joueur possède :

```
id
name
score
level
```

Notre API devra permettre de :

```
Récupérer tous les joueurs
```

```
GET /players
```

Récupérer un joueur

GET /players/{player_id}

Créer un joueur

POST /players

Modifier un joueur

PUT /players/{player_id}

Supprimer un joueur

DELETE /players/{player_id}

9. Pourquoi FastAPI ?

Nous pourrions construire nous-mêmes un serveur HTTP en Python.

Mais il faudrait gérer beaucoup de choses :

- réception des requêtes ;
- routage ;
- parsing du JSON ;
- validation ;
- gestion des erreurs ;
- sérialisation ;
- documentation ;
- etc.

C'est précisément le rôle d'un framework.

Pour notre API, nous utiliserons :

FastAPI

FastAPI est un framework Python permettant de construire des APIs modernes.

Il s'appuie fortement sur :

- les **Type Hints** Python ;
 - **Pydantic** ;
 - un serveur **ASGI** ;
 - **OpenAPI**.
-

10. Installer FastAPI

Créons un environnement virtuel :

```
python -m venv .venv
```

Activation sous Linux / macOS :

```
source .venv/bin/activate
```

Sous Windows :

```
.venv\Scripts\activate
```

Puis :

```
pip install "fastapi[standard]"
```

11. Notre première API

Créons un fichier :

```
main.py
```

Avec seulement quelques lignes :

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Welcome to the Game API"}
```

12. Lancer le serveur

Depuis le terminal :

```
fastapi dev main.py
```

Le serveur démarre localement.

Par défaut :

```
http://127.0.0.1:8000
```

Ouvrons cette adresse dans un navigateur.

Nous obtenons :

```
{
  "message": "Welcome to the Game API"
}
```

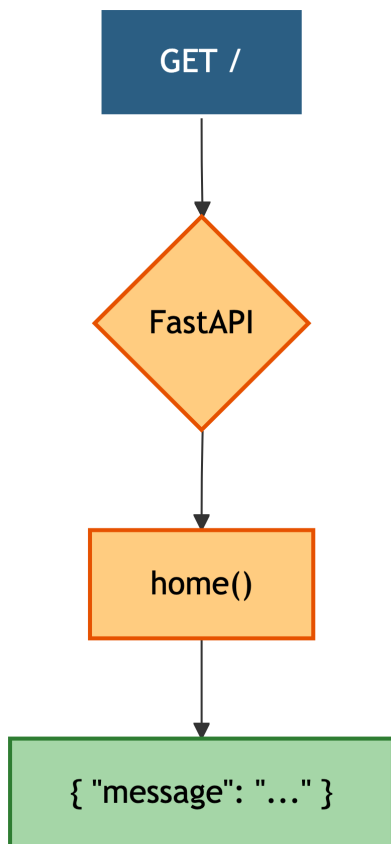
13. Que s'est-il passé ?

Lorsque nous avons écrit :

```
@app.get("/")
```

nous avons indiqué à FastAPI :

Lorsqu'une requête **GET** est envoyée vers **/**, exécute cette fonction.



Cette association entre une URL et une fonction est appelée une **route**.

14. Les routes

Pour manipuler nos données, simulons une petite base de données avec un dictionnaire :

```
PLAYERS = {  
    "1": {"id": 1, "name": "Alice", "score": 1200},  
    "2": {"id": 2, "name": "Bob", "score": 950},  
    "42": {"id": 42, "name": "Zelda", "score": 3000}  
}
```

Une route est définie par :

```
@app.get("/players")  
def get_players():  
    ...
```

On peut lire cela comme :

Pour une requête GET vers `/players`, exécuter `get_players()`.

Exemple :

```
@app.get("/players")  
def get_players():  
    # On retourne la liste de tous les joueurs  
    return list(PLAYERS.values())
```

Tester :

```
http://127.0.0.1:8000/players  
GET /players
```

Résultat :

```
[  
  {  
    "id": 1,  
    "name": "Alice",  
    "score": 1200  
  },  
  {
```

```
        "id": 2,  
        "name": "Bob",  
        "score": 950  
    },  
    {  
        "id": 42,  
        "name": "Zelda",  
        "score": 3000  
    }  
]
```

15. Paramètres de chemin

Comment récupérer uniquement le joueur 42 ?

Nous pouvons utiliser un paramètre variable dans l'URL :

```
@app.get("/players/{player_id}")  
def get_player(player_id):  
    # Par défaut, player_id est du texte (str)  
    return PLAYERS.get(player_id)
```

Une requête :

```
http://127.0.0.1:8000/players/42  
GET /players/42
```

donne :

```
{  
    "id": 42,  
    "name": "Zelda",  
    "score": 3000  
}
```